

PYTHON

DE ZERO AU WEB

Documentation personnalisée

Niveau: Bases -> Développement Web (FastAPI, Django, Flask)

Créé spécialement pour vous | Niveau: Débutant -> Développement Web

SOMMAIRE

#	Chapitre	Contenu cle
1	Les Bases de Python	Variables, types, operateurs, entrees
2	Structures de Controle	If/else, boucles for/while, break/continue
3	Les Fonctions	Def, parametres, retour, portee
4	Structures de Donnees	Listes, tuples, dictionnaires, sets
5	Programmation Orientee Objet	Classes, objets, heritage, methodes
6	Modules et Packages	Import, pip, bibliotheques standard
7	Gestion des Erreurs	Try/except, exceptions, debugging
8	Fichiers et JSON	Lecture/ecriture, format JSON/CSV
9	Introduction a FastAPI	Routes, endpoints, requetes HTTP, Pydantic

Introduction — Pourquoi Python pour le Web ?

Python est aujourd'hui l'un des langages de programmation les plus utilisés dans le monde. Il est simple à lire, puissant, et surtout très demandé dans le développement web moderne.

Votre parcours dans ce guide :

- Maîtriser les bases solides de Python
- Comprendre la logique de programmation
- Construire des APIs web avec FastAPI
- S'ouvrir vers Django et Flask

NOTE

FastAPI est un framework web moderne, ultra-rapide, idéal pour créer des APIs en Python.

Il utilise des concepts Python avancés, c'est pourquoi nous commencerons par les fondamentaux.

Chaque chapitre vous rapproche de votre objectif : créer de vraies applications web !

CH.1 Les Bases de Python

1.1 — Variables et Types de Donnees

Une variable, c'est comme une boîte étiquetée dans laquelle on range une valeur. En Python, pas besoin de déclarer le type à l'avance : Python le devine tout seul !

Les types fondamentaux

# Les principaux types en Python			
nom	= 'Alice'	# str	-> texte (chaîne de caractères)
age	= 25	# int	-> nombre entier
taille	= 1.75	# float	-> nombre decimal
actif	= True	# bool	-> vrai ou faux
# Afficher les valeurs			
print(nom)	# Affiche : Alice		
print(age)	# Affiche : 25		
print(type(age))	# Affiche : <class 'int'>		

TIP

La fonction `type()` permet de connaître le type d'une variable.
Exemple : `type('bonjour')` retourne `<class 'str'>`

Concatenation et f-strings

Pour afficher du texte avec des variables, Python offre les f-strings (la méthode moderne et recommandée) :

prenom = 'Mohamed'
ville = 'Libreville'
age = 22
Methode moderne : f-string (recommandee)
message = f'Bonjour, je suis {prenom}, j\'ai {age} ans et je vis a {ville}.'
print(message)
Affiche : Bonjour, je suis Mohamed, j'ai 22 ans et je vis a Libreville.
On peut aussi faire des calculs dans les f-strings
print(f'Dans 5 ans, j\'aurai {age + 5} ans.')

1.2 — Operateurs

Les opérateurs permettent d'effectuer des calculs et des comparaisons.

Operateur	Description	Exemple	Resultat
+	Addition	5 + 3	8

-	Soustraction	10 - 4	6
*	Multiplication	3 * 4	12
/	Division	10 / 3	3.333...
//	Division entiere	10 // 3	3
%	Modulo (reste)	10 % 3	1
**	Puissance	2 ** 8	256

1.3 — Entree utilisateur

La fonction `input()` permet de demander une information a l'utilisateur :

```
# Demander un prenom
prenom = input('Entrez votre prenom : ')
print(f'Bonjour, {prenom} !')

# ATTENTION : input() retourne toujours un texte (str)
# Pour un nombre, il faut convertir :
age_str = input('Entrez votre age : ')
age = int(age_str) # Convertir en entier
print(f'Dans 10 ans, vous aurez {age + 10} ans.')
```



`input()` retourne TOUJOURS une chaine de caracteres (str), meme si l'utilisateur tape un nombre.
Il faut utiliser `int()` pour convertir en entier, ou `float()` pour un nombre decimal.

EXERCICES — Chapitre 1

★ Exercice FACILE — Carte de visite

Creez un programme qui demande a l'utilisateur : son prenom, son age, et sa ville.

Puis affichez une phrase comme : 'Je m'appelle Koffi, j'ai 20 ans et j'habite a Libreville.'

Indice : Utilisez `input()` pour les entrees et une f-string pour l'affichage.

★★ Exercice MOYEN — Calculatrice simple

Creez un programme qui :

- 1) Demande deux nombres a l'utilisateur
- 2) Affiche leur somme, difference, produit et quotient
- 3) Affiche aussi le reste de la division (modulo) et la puissance

Indice : Pensez a convertir les entrees avec `float()` pour gerer les decimaux.

★★★ Exercice DIFFICILE — Convertisseur universe

Creez un programme qui :

- 1) Demande une temperature en Celsius
- 2) La convertit en Fahrenheit (formule : $F = C * 9/5 + 32$) et en Kelvin ($K = C + 273.15$)
- 3) Affiche les trois valeurs avec 2 decimales (utilisez `:.2f` dans la f-string)
- 4) Indique si la temperature est : glaciale (<0), froide (0-15), moderee (15-25), chaude (>25)

Indice : Cherchez comment afficher 2 decimales dans une f-string avec `:.2f`

PROJET DE FIN DE CHAPITRE — Mini-Profil Utilisateur**Description**

Creez un programme interactif qui construit un profil utilisateur complet et le presente de facon stylisee dans le terminal.

Objectifs

1. Demander : prenom, nom, age, ville, metier/etudes
2. Calculer l'annee de naissance (utilisez 2025 comme annee actuelle)
3. Afficher un encadre textuel avec toutes les informations
4. Calculer et afficher l'age dans 5, 10 et 20 ans
5. Afficher un message personnalise selon l'age (mineur, jeune adulte, adulte, senior)

Bonus

Ajoutez la validation : si l'utilisateur entre un age negatif ou superieur a 120, affichez un message d'erreur.

CH.2 Structures de Controle

2.1 — Les Conditions (if / elif / else)

Les conditions permettent a votre programme de prendre des decisions. Imaginez un agent de securite : 'Si tu as plus de 18 ans, tu entres. Sinon, tu restes dehors !'

```
age = 17

if age >= 18:
    print('Vous etes majeur, bienvenue !')
elif age >= 15:
    print('Vous etes presque majeur...')
else:
    print('Acces refuse, vous etes mineur.')

# Operateurs de comparaison : ==, !=, >, <, >=, <=
# Operateurs logiques      : and, or, not

# Exemple combine
heure = 14
if heure >= 8 and heure < 12:
    print('Matin')
elif heure >= 12 and heure < 18:
    print('Apres-midi')
else:
    print('Soiree ou nuit')
```

NOTE

En Python, l'INDENTATION (les espaces en debut de ligne) est OBLIGATOIRE et fait partie de la syntaxe. Utilisez toujours 4 espaces (ou une tabulation) pour indenter votre code.

2.2 — La Boucle for

La boucle for permet de repeter une action pour chaque element d'une sequence. C'est comme dire : 'Pour chaque etudiant de la classe, dis bonjour.'

```
# Parcourir une liste de noms
etudiants = ['Alice', 'Bob', 'Charlie', 'Diana']

for etudiant in etudiants:
    print(f'Bonjour, {etudiant} !')

# Utiliser range() pour compter
for i in range(5):          # 0, 1, 2, 3, 4
    print(f'Tour numero {i}')

for i in range(1, 11):      # 1, 2, ..., 10
```

<code>print(f'{i} x 5 = {i*5}')</code>
<code>for i in range(0, 20, 2):</code> <code># 0, 2, 4, ..., 18 (pas de 2)</code>
<code>print(i, end=' ')</code> <code># end=' ' evite le retour a la ligne</code>

2.3 — La Boucle while

La boucle while continue TANT QUE une condition est vraie. C'est comme : 'Continue a courir tant que tu as de l'energie.'

<code># Compter jusqu'a 5</code>
<code>compteur = 0</code>
<code>while compteur < 5:</code>
<code>print(f'Compteur : {compteur}')</code>
<code>compteur += 1</code> <code># Equivalent a : compteur = compteur + 1</code>
<code># Boucle avec input (tres utile !)</code>
<code>reponse = ''</code>
<code>while reponse != 'quitter':</code>
<code>reponse = input('Tapez quelque chose (ou "quitter" pour sortir) : ')</code>
<code>print(f'Vous avez dit : {reponse}')</code>
<code>print('Au revoir !')</code>

2.4 — break, continue et pass

<code># break : sortir de la boucle immediatement</code>
<code>for i in range(10):</code>
<code>if i == 5:</code>
<code>break</code> <code># Arrete la boucle quand i vaut 5</code>
<code>print(i)</code> <code># Affiche : 0 1 2 3 4</code>
<code># continue : passer a l'iteration suivante</code>
<code>for i in range(10):</code>
<code>if i % 2 == 0:</code> <code># Si i est pair</code>
<code>continue</code> <code># Sauter cet element</code>
<code>print(i)</code> <code># Affiche seulement les impairs : 1 3 5 7 9</code>
<code># pass : ne rien faire (placeholder)</code>
<code>for i in range(5):</code>
<code>pass</code> <code># Rien pour l'instant</code>

EXERCICES — Chapitre 2

★ Exercice FACILE — Table de multiplication

Demandez un nombre a l'utilisateur et affichez sa table de multiplication de 1 a 10.

Exemple pour 7 : '7 x 1 = 7', '7 x 2 = 14', etc.

Indice : Utilisez une boucle for avec `range(1, 11)`.

★★ Exercice MOYEN — Jeu du nombre secret

Creez un jeu ou l'ordinateur a choisi le nombre 42 (codez-le en dur).

L'utilisateur doit le deviner. A chaque essai, dites si c'est 'trop grand', 'trop petit' ou 'gagne !'.

Comptez le nombre d'essais et affichez-le a la fin.

Indice : Utilisez une boucle while et break quand le joueur trouve. Utilisez `int(input(...))`.

★★★ Exercice DIFFICILE — Analyse de donnees

Demandez a l'utilisateur de saisir des notes (entre 0 et 20) une par une.

Il s'arrete en tapant -1.

A la fin, affichez : la moyenne, la note la plus haute, la note la plus basse,

le nombre de notes saisies, et le pourcentage de notes superieures a la moyenne.

Ignorez les notes invalides (hors [0,20]) avec un message d'avertissement.

Indice : Utilisez une liste pour stocker les notes, et `sum()/len()` pour la moyenne.

PROJET DE FIN DE CHAPITRE — Quiz Interactif

Description

Creez un quiz a choix multiples en Python avec au moins 5 questions sur un sujet de votre choix (Python, geographie, culture generale...).

Objectifs

6. Stocker les questions, options et reponses correctes dans des variables
7. Afficher chaque question avec ses choix (A, B, C, D)
8. Compter le score de l'utilisateur
9. Afficher le resultat final avec un message selon le score (felicitations, encouragements...)
10. Permettre de rejouer a la fin

Bonus

Melangez les questions aleatoirement avec le module `random` et `random.shuffle()`.

CH.3 Les Fonctions

3.1 — Créer et Appeler une Fonction

Une fonction est un bloc de code réutilisable. Au lieu de réécrire le même code plusieurs fois, vous le mettez dans une fonction et vous l'appellez quand vous en avez besoin. C'est comme une recette de cuisine : vous l'écrivez une fois, vous la suivez autant de fois que vous voulez.

```
# Définir une fonction
def saluer(prenom):
    message = f'Bonjour, {prenom} ! Bienvenue en Python.'
    return message

# Appeler la fonction
resultat = saluer('Amina')
print(resultat)  # Affiche : Bonjour, Amina ! Bienvenue en Python.

# Appeler plusieurs fois avec des arguments différents
print(saluer('Jean'))    # Bonjour, Jean !
print(saluer('Fatou'))   # Bonjour, Fatou !
```

3.2 — Paramètres par défaut et arguments nommés

```
# Paramètre avec valeur par défaut
def presenter(nom, age=18, ville='Libreville'):
    return f'{nom}, {age} ans, de {ville}'

print(presenter('Ali'))           # Ali, 18 ans, de Libreville
print(presenter('Sara', 25))      # Sara, 25 ans, de Libreville
print(presenter('Omar', 30, 'Paris')) # Omar, 30 ans, de Paris

# Arguments nommés (ordre libre !)
print(presenter(ville='Dakar', nom='Lena')) # Lena, 18 ans, de Dakar
```

3.3 — Fonctions avec plusieurs valeurs de retour

```
# Retourner plusieurs valeurs (en fait un tuple)
def calculer(a, b):
    somme = a + b
    difference = a - b
    produit = a * b
    return somme, difference, produit

# Dépackager les résultats
s, d, p = calculer(10, 3)
print(f'Somme: {s}, Difference: {d}, Produit: {p}')
# Somme: 13, Difference: 7, Produit: 30
```

3.4 — Portee des variables (scope)

Les variables creees dans une fonction ne sont visibles QUE dans cette fonction :

```
x = 10 # Variable globale (accessible partout)

def ma_fonction():
    y = 20 # Variable locale (seulement dans la fonction)
    print(x) # OK : peut lire la variable globale
    print(y) # OK : variable locale

ma_fonction()
print(x) # OK
# print(y) # ERREUR ! y n'existe pas hors de la fonction
```

!

Evitez au maximum d'utiliser 'global' pour modifier des variables globales dans une fonction.

C'est une mauvaise pratique qui rend le code difficile a maintenir.

Preferiez toujours passer les valeurs en parametre et retourner le resultat.

EXERCICES — Chapitre 3

★ Exercice FACILE — Calculatrice fonctionnelle

Ecrivez 4 fonctions : additionner(a, b), soustraire(a, b), multiplier(a, b), diviser(a, b).

La fonction diviser doit retourner un message d'erreur si b vaut 0.

Testez chaque fonction avec au moins 2 appels.

Indice : Utilisez une condition `if b == 0` dans la fonction diviser.

★★ Exercice MOYEN — Validateur de mot de passe

Creez une fonction `valider_mdp(mdp)` qui verifie qu'un mot de passe :

- Contient au moins 8 caracteres
- Contient au moins une majuscule
- Contient au moins un chiffre

La fonction retourne `True` si valide, `False` sinon.

Creez aussi une fonction `afficher_force(mdp)` qui dit : Faible, Moyen ou Fort.

Indice : Utilisez les methodes : `len()`, `any()`, `str.isupper()`, `str.isdigit()`. Cherchez comment les utiliser !

★★★ Exercice DIFFICILE — Mini-framework de tests

Creez les fonctions suivantes :

- `est_palindrome(texte)` : retourne `True` si le texte se lit pareil dans les deux sens
- `compter_mots(texte)` : retourne un dictionnaire `{mot: nombre_occurrences}`
- `inverser_mots(texte)` : retourne la phrase avec les mots dans l'ordre inverse
- Creez une fonction `tester(nom_test, resultat, attendu)` qui affiche OK ou ECHEC
- Testez chaque fonction avec au moins 3 cas (y compris des cas limites : chaine vide, un seul mot...)

Indice : Utilisez `texte.lower().split()` pour decoupe et `texte[::-1]` pour inverser une chaine.

PROJET DE FIN DE CHAPITRE — Bibliotheque de Calculs Mathematiques

Description

Creez un module Python contenant une collection de fonctions mathematiques utiles, utilisable comme une vraie mini-bibliotheque.

Objectifs

11. Fonction `factorielle(n)` recursive et iterative
12. Fonction `est_premier(n)` qui teste si un nombre est premier
13. Fonction `fibonacci(n)` qui retourne les `n` premiers termes
14. Fonction `pgcd(a, b)` (Plus Grand Commun Diviseur) avec l'algorithme d'Euclide
15. Un menu interactif qui permet de choisir et tester chaque fonction

Bonus

Ajoutez de la documentation a chaque fonction avec des docstrings (ex: `def ma_func():`
"`Description de la fonction`").

CH.4 Structures de Donnees

4.1 — Les Listes

Une liste est une collection ORDONNEE et MODIFIABLE d'elements. C'est la structure la plus utilisee en Python.

```
# Creer une liste
fruits = ['pomme', 'banane', 'mangue', 'orange']

# Acceder par index (commence a 0 !)
print(fruits[0])    # pomme
print(fruits[-1])   # orange (dernier element)

# Modifier
fruits[1] = 'ananas'

# Methodes essentielles
fruits.append('fraise')    # Ajouter en fin
fruits.insert(0, 'citron') # Insérer a la position 0
fruits.remove('mangue')    # Supprimer par valeur
popped = fruits.pop()      # Supprimer et retourner le dernier

# Slice : extraire une sous-liste
trois_premiers = fruits[0:3] # Index 0, 1, 2
inverses       = fruits[::-1] # Liste inversee

print(len(fruits))    # Nombre d'elements
print('banane' in fruits) # True ou False
```

4.2 — Les Dictionnaires

Un dictionnaire stocke des donnees sous forme cle: valeur. C'est l'equivalent d'un vrai dictionnaire : tu cherches un mot (cle), tu trouves sa definition (valeur).

```
# Creer un dictionnaire
utilisateur = {
    'nom':    'Kofi',
    'age':    28,
    'email':  'kofi@example.com',
    'actif':  True
}

# Acceder
print(utilisateur['nom'])    # Kofi
print(utilisateur.get('ville', 'Non renseigne')) # Valeur par default

# Modifier / Ajouter
utilisateur['age'] = 29
utilisateur['ville'] = 'Accra'
```

```
# Parcourir
for cle, valeur in utilisateur.items():
    print(f'{cle}: {valeur}')

# Verifier l'existence d'une cle
if 'email' in utilisateur:
    print('Email present')
```

NOTE

Les dictionnaires sont TRES importants pour FastAPI !
 Les donnees JSON (que vous utiliserez dans vos APIs) sont directement converties en dictionnaires Python.
 Exemple : `{'nom': 'Alice', 'age': 25}` en Python = `{"nom": "Alice", "age": 25}` en JSON.

4.3 — Tuples et Sets

```
# Tuple : liste IMMuable (ne peut pas etre modifiee)
coordonnees = (48.8566, 2.3522) # Paris
x, y = coordonnees # Depackager
print(x) # 48.8566

# Set : collection SANS DOUBLON et NON ORDONNEE
pays_visites = {'France', 'Maroc', 'Gabon', 'France'}
print(pays_visites) # {'France', 'Maroc', 'Gabon'} - pas de doublon !

pays_visites.add('Senegal')

# Operations sur les sets
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a & b) # Intersection : {3, 4}
print(a | b) # Union : {1, 2, 3, 4, 5, 6}
print(a - b) # Difference : {1, 2}
```

4.4 — Comprehensions de listes

Une facon elegante et pythonique de creer des listes en une seule ligne :

```
# Version classique
carres = []
for i in range(10):
    carres.append(i**2)

# Version avec comprehension (recommandee)
carres = [i**2 for i in range(10)]
print(carres) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Avec condition
pairs = [i for i in range(20) if i % 2 == 0]
print(pairs) # [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

```
# Comprehension de dictionnaire
mots = ['python', 'web', 'api']
longueurs = {mot: len(mot) for mot in mots}
print(longueurs)  # {'python': 6, 'web': 3, 'api': 3}
```

EXERCICES — Chapitre 4

★ Exercice FACILE — Gestion d'une liste d'achats

Creez une liste d'achats vide.

Ecrivez un programme qui : ajoute des articles (input), affiche la liste, supprime un article, affiche le nombre d'articles.

L'utilisateur s'arrete en tapant 'fin'.

Indice : Utilisez `.append()` pour ajouter et `.remove()` pour supprimer.

★★ Exercice MOYEN — Carnet de contacts

Creez un systeme de contacts utilisant un dictionnaire de dictionnaires.

Chaque contact a : nom, telephone, email.

Implementez : ajouter un contact, afficher tous les contacts, rechercher par nom, supprimer un contact.

Affichez les contacts triees par nom.

Indice : Utilisez un dict principal ou la cle est le nom. Triez avec `sorted(contacts.keys())`.

★★★ Exercice DIFFICILE — Statistiques avancees

Generez une liste de 50 nombres aleatoires entre 1 et 100 (utilisez `random.randint`).

Calculez SANS utiliser les fonctions `sum`/`max`/`min` built-in :

- la somme, la moyenne, le minimum, le maximum
- la mediane (valeur centrale apres tri)
- la variance et l'ecart-type
- les nombres qui apparaissent plus d'une fois (doublons)

Affichez un histogramme en texte par tranches de 10 (0-9, 10-19, etc.)

Indice : Pour l'histogramme, utilisez un dictionnaire avec les tranches comme cles.

PROJET DE FIN DE CHAPITRE — Gestionnaire de Bibliotheque

Description

Creez un systeme de gestion d'une bibliotheque simple en ligne de commande.

Objectifs

16. Stocker les livres dans une liste de dictionnaires (titre, auteur, annee, disponible)
17. Menu : ajouter un livre, lister tous les livres, rechercher par titre ou auteur
18. Emprunt : marquer un livre comme indisponible avec le nom de l'emprunteur
19. Retour : rendre un livre disponible
20. Statistiques : nombre de livres, disponibles vs empruntes, auteur le plus present

Bonus

Sauvegardez la bibliotheque dans un fichier JSON (voir Chapitre 8) pour persister les donnees entre les executions.

CH.5 Programmation Orientee Objet (POO)

5.1 — Classes et Objets

La POO est une facon d'organiser votre code autour d'objets du monde reel. Pensez a une voiture : elle a des caracteristiques (marque, couleur, vitesse) et des actions (demarrer, accelerer, freiner). En Python, on cree une classe pour definir ce modele, et des objets (instances) pour creer des voitures reelles.

```
class Utilisateur:
    # __init__ est le constructeur : appele quand on cree un objet
    def __init__(self, nom, email, age):
        self.nom = nom      # Attributs de l'objet
        self.email = email
        self.age = age
        self.actif = True   # Valeur par default

    # Methode : action que l'objet peut faire
    def sePresenter(self):
        return f'Je suis {self.nom}, {self.age} ans ({self.email})'

    def desactiver(self):
        self.actif = False
        print(f'Le compte de {self.nom} a ete desactive.')

    def __str__(self):      # Representation en chaine de caracteres
        statut = 'actif' if self.actif else 'inactif'
        return f'Utilisateur({self.nom}, {statut})'

# Creer des objets (instances de la classe)
user1 = Utilisateur('Alice', 'alice@example.com', 28)
user2 = Utilisateur('Bob', 'bob@example.com', 35)

print(user1.sePresenter())
print(user2)
user1.desactiver()
```

5.2 — Heritage

L'heritage permet a une classe de heriter des caracteristiques d'une autre. C'est comme dire : un Admin EST un Utilisateur, mais avec des pouvoirs supplementaires.

```
class Admin(Utilisateur):    # Admin herite de Utilisateur
    def __init__(self, nom, email, age, niveau_acces):
        super().__init__(nom, email, age) # Appeler le constructeur parent
        self.niveau_acces = niveau_acces
        self.permissions = []

    def ajouter_permission(self, permission):
```

```

        self.permissions.append(permission)
        print(f'Permission {permission} ajoutée a {self.nom}')

# Surcharge de methode
def sePresenter(self):
    base = super().sePresenter() # Appeler la methode du parent
    return f'{base} [Admin niveau {self.niveau_acces}]'

admin = Admin('Superadmin', 'admin@example.com', 30, 5)
admin.ajouter_permission('supprimer_utilisateur')
print(admin.sePresenter())

```

NOTE

La POO est FONDAMENTALE pour FastAPI et Django.
 Dans FastAPI, les modeles Pydantic sont des classes.
 Dans Django, les modeles de base de donnees sont des classes qui heritent de `models.Model`.
 Maitriser la POO maintenant vous facilitera enormement la vie par la suite !

EXERCICES — Chapitre 5

★ Exercice FACILE — Classe Voiture

Creez une classe Voiture avec : marque, modele, annee, kilometrage.

Methodes : `demarrer()`, `rouler(km)` qui ajoute au kilometrage, `afficher_infos()`.

Creez 3 voitures differentes et testez toutes les methodes.

Indice : *N'oubliez pas self dans chaque methode et self.attribut pour les attributs.*

★★ Exercice MOYEN — Systeme Bancaire

Creez une classe CompteBancaire avec : titulaire, solde (default 0), numero de compte.

Methodes : `deposer(montant)`, `retirer(montant)`, `virer(montant, autre_compte)`, `afficher_historique()`.

Creez une classe CompteEpargne qui herite de CompteBancaire et ajoute un taux d'interet et une methode `calculer_interet()`.

Indice : *Generez le numero de compte automatiquement avec un compteur de classe (variable de classe).*

★★★ Exercice DIFFICILE — Jeu RPG

Creez un mini-RPG avec : classe Personnage (nom, vie, attaque, defense),

classe Guerrier et classe Mage qui heritent de Personnage avec des competences speciales.

Methode `attaquer(cible)` : la cible perd (attaque - defense) points de vie.

Methode `est_vivant()`, `afficher_statut()`.

Simulez un combat tour par tour entre un Guerrier et un Mage avec une boucle.

Indice : Utilisez `random.randint` pour ajouter de l'aleatoire dans les degats.

PROJET DE FIN DE CHAPITRE — Systeme de Gestion Scolaire

Description

Creez un systeme de gestion d'ecole complet en POO.

Objectifs

21. Classe Personne (base) : nom, prenom, date_naissance
22. Classe Etudiant (herite Personne) : numero etudiant, notes, methodes: ajouter_note, calculer_moyenne, obtenir_mention
23. Classe Professeur (herite Personne) : matiere, salaire
24. Classe Cours : nom, professeur, liste etudiants, methodes: inscrire, afficher_classement
25. Programme principal avec menu pour gerer tout le systeme

Bonus

Ajoutez la serialisation : sauvegardez et chargez les donnees depuis un fichier JSON.

CH.6 Modules, Packages et pip

6.1 — Importer des modules

Python est livree avec une enorme bibliotheque standard. On peut aussi installer des packages tiers avec pip.

Importer un module entier
import math
print(math.sqrt(16)) # 4.0
print(math.pi) # 3.14159...
Importer une fonction specifique
from math import sqrt, ceil, floor
print(sqrt(25)) # 5.0
Importer avec un alias (tres courant en data science)
import random as rd
print(rd.randint(1, 10))
Modules utiles a connaitre
import os # Systeme de fichiers
import sys # Systeme Python
import datetime # Dates et heures
import json # JSON
import re # Expressions regulieres
import time # Temporisation

6.2 — pip : installer des packages

pip est le gestionnaire de packages Python. Il permet d'installer des bibliotheques tierces.

Dans votre terminal (pas dans Python) :
Installer un package
pip install requests
pip install fastapi
pip install uvicorn
Installer plusieurs packages
pip install fastapi uvicorn pydantic
Voir les packages installes
pip list
Creer un fichier de dependances (tres important !)
pip freeze > requirements.txt
Installer depuis un fichier de dependances
pip install -r requirements.txt

6.3 — Creer votre propre module

Un fichier .py est automatiquement un module importable :

Fichier : utils.py
def formater_nom(prenom, nom):
return f'{prenom.capitalize()} {nom.upper()}'
def calculer_tva(prix, taux=0.20):
return round(prix * (1 + taux), 2)
PI = 3.14159
=====
Fichier : main.py (meme dossier)
import utils
print(utils.formater_nom('alice', 'dupont')) # Alice DUPONT
print(utils.calculer_tva(100)) # 120.0
print(utils.PI)

EXERCICES — Chapitre 6

★ Exercice FACILE — Explorez la bibliotheque standard

En utilisant uniquement les modules standard (os, datetime, random, math) :

Affichez la date et l'heure actuelles, le dossier de travail actuel, un mot de passe aleatoire de 8 caracteres (chiffres + lettres), et le cosinus de 90 degrees.

Indice : Cherchez `datetime.now()`, `os.getcwd()`, `random.choice()`, `math.cos()`. Utilisez la documentation !

★★ Exercice MOYEN — Module utilitaire personnel

Creez un fichier `mon_utils.py` contenant au moins 5 fonctions utiles :

`formater_date(date)`, `est_email_valide(email)`, `generer_mot_de_passe(longueur)`, `tronquer_texte(texte, max)`, `capitaliser_mots(phrase)`.

Dans un fichier `main.py`, importez et testez chaque fonction.

Indice : Utilisez le module `re` pour la validation email et `random` pour le mot de passe.

★★★ Exercice DIFFICILE — Analyseur de fichiers

Creez un programme qui analyse un fichier texte passe en argument (`sys.argv[1]`) :

Nombre de lignes, mots, caracteres, les 10 mots les plus frequents (excluant les mots de liaison : le, la, les, de, du, un, une, et, ou...),

Duree de lecture estimee (250 mots/minute), et generez un rapport dans un fichier .txt.

Indice : Utilisez `collections.Counter` pour compter les occurrences. C'est la facon pythonique !

PROJET DE FIN DE CHAPITRE — Package Utilitaire Complet

Description

Creez un package Python structuree (un dossier avec `__init__.py`) contenant plusieurs modules.

Objectifs

- 26. `mon_package/texte.py` : fonctions de traitement de texte
- 27. `mon_package/maths.py` : fonctions mathematiques avancees
- 28. `mon_package/dates.py` : utilitaires pour les dates
- 29. `mon_package/fichiers.py` : lecture/ecriture de fichiers
- 30. Un fichier `README.md` documentant chaque fonction

Bonus

Publiez votre package localement avec `pip install -e .` et testez-le depuis n'importe quel dossier.

CH.7 Gestion des Erreurs et Exceptions

7.1 — Try / Except

Les erreurs en Python sont appelees 'exceptions'. Sans gestion des erreurs, votre programme s'arrete brutalement. Avec try/except, vous pouvez les attraper et les gerer proprement.

```
# Sans gestion : le programme plante
# print(10 / 0) # ZeroDivisionError !

# Avec try/except
try:
    nombre = int(input('Entrez un nombre : '))
    resultat = 100 / nombre
    print(f'100 / {nombre} = {resultat}')

except ValueError:
    print('Erreur : vous n\'avez pas entre un nombre valide !')

except ZeroDivisionError:
    print('Erreur : impossible de diviser par zero !')

except Exception as e:
    print(f'Erreur inattendue : {e}')

else:
    print('Tout s\'est passe sans erreur !')

finally:
    print('Ce bloc s\'execute TOUJOURS (erreur ou non).')
```

7.2 — Exceptions personnalisees

Vous pouvez creer vos propres types d'erreurs pour rendre votre code plus expressif :

```
class AgeInvalideError(Exception):
    def __init__(self, age):
        self.age = age
        super().__init__(f'L\'age {age} est invalide (doit etre entre 0 et 150)')

class SoldeInsuffisantError(Exception):
    def __init__(self, solde, montant):
        super().__init__(f'Solde insuffisant : {solde} < {montant}')
```

```
# Lever une exception
def creer_utilisateur(nom, age):
    if not 0 <= age <= 150:
        raise AgeInvalideError(age)
    return {'nom': nom, 'age': age}
```

```
# Utilisation
try:
    user = creer_utilisateur('Alice', 200)
except AgeInvalideError as e:
    print(f'Erreur de creation : {e}')
```

NOTE

Dans FastAPI et Django, la gestion des erreurs est CRITIQUE.
Les erreurs HTTP (400, 404, 500...) sont des exceptions que vous leverez dans vos routes.
Bien gerer les erreurs = code robuste et APIs fiables.

EXERCICES — Chapitre 7

★ Exercice FACILE — Calculatrice robuste

Reprenez votre calculatrice du chapitre 3 et ajoutez une gestion complete des erreurs :

- Si l'utilisateur entre autre chose qu'un nombre -> ValueError
- Si division par zero -> ZeroDivisionError
- Le programme ne doit JAMAIS planter, il doit toujours afficher un message propre.

Indice : Enveloppez chaque `int(input())` dans un bloc `try/except ValueError`.

★★ Exercice MOYEN — Valideur de donnees

Creez des exceptions personnalisees : `EmailInvalideError`, `TelephoneInvalideError`, `AgeInvalideError`.

Creez une fonction `valider_profil(nom, email, telephone, age)` qui leve l'exception appropriee.

Ecrivez une fonction `creer_profil()` qui appelle `valider_profil` et gere toutes les exceptions.

Indice : Un email valide contient exactement un @ et un point apres le @.

★★★ Exercice DIFFICILE — Systeme de retry avec logging

Creez une fonction `appel_api_simule(url)` qui simule un appel reseau :

- Genere aleatoirement une `TimeoutError` (30% de chance), `ConnectionError` (20%) ou reussit (50%)
- Creez une fonction `avec_retry(func, max_tentatives=3, delai=1)` qui reessaie automatiquement
- Loggez chaque tentative avec le module logging (date, tentative, succes/echec)
- Apres le nombre max de tentatives, levez une erreur finale personnalisee

Indice : Utilisez `time.sleep(delai)` pour attendre entre les tentatives et import logging pour les logs.

PROJET DE FIN DE CHAPITRE — API de Validation Robuste

Description

Creez un systeme complet de validation et de nettoyage de donnees utilisateur, comme ce qu'on trouve dans les vraies applications.

Objectifs

31. Definir au moins 6 exceptions personnalisees specifiques
32. Valider : email, mot de passe fort, numero de telephone, date de naissance, URL, code postal
33. Creer une classe FormValideur qui valide un formulaire complet et retourne tous les erreurs d'un coup
34. Ajouter des logs avec le module logging (niveaux INFO, WARNING, ERROR)
35. Gerer le cas ou plusieurs champs sont invalides (ne pas s'arreter au premier)

Bonus

C'est exactement ce que fait Pydantic dans FastAPI ! Vous comprendrez mieux le prochain chapitre.

CH.8 Fichiers, JSON et Persistance des Donnees

8.1 — Lire et Ecrire des Fichiers

```
# Ecrire dans un fichier
with open('notes.txt', 'w', encoding='utf-8') as f:
    f.write('Ligne 1\n')
    f.write('Ligne 2\n')

# Lire un fichier entier
with open('notes.txt', 'r', encoding='utf-8') as f:
    contenu = f.read()
    print(contenu)

# Lire ligne par ligne
with open('notes.txt', 'r', encoding='utf-8') as f:
    for ligne in f:
        print(ligne.strip()) # .strip() supprime le \n

# Modes d'ouverture
# 'r' : lecture seule (default)
# 'w' : ecriture (ecrase le fichier existant)
# 'a' : ajout en fin de fichier
# 'r+' : lecture et ecriture
```

8.2 — JSON : Le Format du Web

JSON est le format universel pour echanger des donnees sur le web. Toutes les APIs (y compris celles que vous ferez avec FastAPI) utilisent JSON.

```
import json

# Python dict -> JSON (serialisation)
utilisateur = {
    'nom': 'Alice',
    'age': 28,
    'competences': ['Python', 'FastAPI', 'SQL'],
    'adresse': {'ville': 'Paris', 'pays': 'France'}
}

# Convertir en chaine JSON
json_str = json.dumps(utilisateur, indent=4, ensure_ascii=False)
print(json_str)

# Sauvegarder dans un fichier
with open('utilisateur.json', 'w', encoding='utf-8') as f:
    json.dump(utilisateur, f, indent=4, ensure_ascii=False)

# Lire depuis un fichier JSON
```

```
with open('utilisateur.json', 'r', encoding='utf-8') as f:
    data = json.load(f)

print(data['nom'])           # Alice
print(data['competences'][0]) # Python
print(data['adresse']['ville']) # Paris
```

TIP

ensure_ascii=False permet d'ecrire les accents directement (e, a, u...) au lieu de \u00e9.
 indent=4 rend le JSON lisible et bien formate.
 json.dumps() -> chaine de caracteres (s = string).
 json.dump() -> ecrit dans un fichier (sans s).

EXERCICES — Chapitre 8

★ Exercice FACILE — Journal personnel

Creez un programme de journal intime :

L'utilisateur peut ecire une entree (avec la date automatique), lire toutes les entrees anterieures, et quitter.

Chaque entree est sauvegardee dans un fichier journal.txt avec la date.

Indice : Utilisez `datetime.now().strftime('%Y-%m-%d %H:%M')` pour formater la date.

★★ Exercice MOYEN — Base de donnees JSON

Creez un mini-systeme CRUD (Create, Read, Update, Delete) pour des produits.

Chaque produit a : id (auto-incremente), nom, prix, quantite.

Toutes les donnees sont persistees dans un fichier produits.json.

Implementez : ajouter, lister, modifier le prix, supprimer par id.

Indice : Chargez le JSON au demarrage, modifiez en memoire, sauvegardez apres chaque operation.

★★★ Exercice DIFFICILE — Analyseur de logs

Generez d'abord un faux fichier de logs (1000 lignes) avec des entrees comme :

2025-01-15 14:32:01 ERROR /api/users Connection timeout

Parsez ce fichier et produisez un rapport JSON contenant :

Nombre d'erreurs par type (INFO, WARNING, ERROR), les 10 routes les plus touchees par des erreurs,

Tableau horaire des erreurs, et les erreurs critiques (plus de 5 fois la meme).

Indice : Utilisez `re.findall()` pour extraire les parties du log avec des regex.

PROJET DE FIN DE CHAPITRE — Application de Gestion de Budget

Description

Creez une application complete de suivi budgetaire avec persistance JSON.

Objectifs

- 36. Ajouter des transactions (revenus et depenses) avec categorie, montant, date, description
- 37. Afficher le solde actuel, les depenses par categorie, l'historique
- 38. Generer un rapport mensuel (revenus, depenses, economies, graphique ASCII)
- 39. Fixer des budgets par categorie et alerter quand depasse
- 40. Exporter les donnees en CSV

Bonus

Ajoutez une interface avec des couleurs dans le terminal en utilisant le module colorama (pip install colorama).

CH.9 Introduction a FastAPI — Votre Premiere API Web

9.1 — Qu'est-ce qu'une API ?

Une API (Application Programming Interface) est une interface qui permet a deux programmes de communiquer. Quand vous utilisez une application mobile pour commander de la nourriture, l'app envoie une requete a une API qui gere les commandes. FastAPI vous permet de creer ce genre de systeme en Python.

NOTE

HTTP : le protocole de communication du Web. Votre navigateur utilise HTTP.
 GET : pour lire des donnees (ex: afficher une liste d'utilisateurs)
 POST : pour creer des donnees (ex: inscrire un nouvel utilisateur)
 PUT : pour modifier des donnees (ex: mettre a jour un profil)
 DELETE : pour supprimer des donnees (ex: supprimer un compte)

9.2 — Installation et Premier Serveur

```
# Dans le terminal : installer FastAPI et Uvicorn
pip install fastapi uvicorn

# Fichier : main.py
from fastapi import FastAPI

app = FastAPI(title='Ma Premiere API', version='1.0')

@app.get('/')          # Decorateur de route
def accueil():
    return {'message': 'Bonjour depuis FastAPI !', 'statut': 'ok'}

@app.get('/bonjour/{prenom}')
def saluer(prenom: str):
    return {'message': f'Bonjour, {prenom} !', 'prenom': prenom}

# Lancer le serveur (dans le terminal) :
# uvicorn main:app --reload
# Ouvrez : http://localhost:8000
# Documentation auto : http://localhost:8000/docs
```

9.3 — Pydantic : Valider les Donnees

Pydantic permet de definir des modeles de donnees avec validation automatique. C'est la que votre apprentissage de la POO et des exceptions va tout prendre son sens !

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, EmailStr
from typing import Optional, List
```

```

app = FastAPI()

# Modele Pydantic (classe qui herite de BaseModel)
class Utilisateur(BaseModel):
    nom: str
    email: str
    age: int
    ville: Optional[str] = None # Champ optionnel

class UtilisateurReponse(Utilisateur):
    id: int

# Base de donnees simulee
db_utilisateurs = []
compteur_id = 0

@app.post('/utilisateurs/', response_model=UtilisateurReponse)
def creer_utilisateur(user: Utilisateur):
    global compteur_id
    compteur_id += 1
    nouvel_user = {'id': compteur_id, **user.dict()}
    db_utilisateurs.append(nouvel_user)
    return nouvel_user

@app.get('/utilisateurs/', response_model>List[UtilisateurReponse])
def lister_utilisateurs():
    return db_utilisateurs

@app.get('/utilisateurs/{user_id}')
def obtenir_utilisateur(user_id: int):
    for user in db_utilisateurs:
        if user['id'] == user_id:
            return user
    raise HTTPException(status_code=404, detail='Utilisateur non trouve')

```

TIP

FastAPI genere automatiquement une documentation interactive a /docs (Swagger UI). Vous pouvez tester vos endpoints directement depuis votre navigateur !
La validation Pydantic est automatique : si le JSON envoye ne correspond pas au modele, FastAPI retourne une erreur 422.

EXERCICES — Chapitre 9

★ Exercice FACILE — API Hello World

Creez une API FastAPI avec 3 routes GET :

GET / -> message de bienvenue avec nom de l'API et version

GET /heure -> retourne l'heure actuelle en JSON

GET /hello/{nom} -> retourne un salut personnalise

Testez avec la documentation interactive /docs.

Indice : Importez `datetime` pour l'heure. Retournez toujours des dictionnaires.

★★ Exercice MOYEN — API CRUD Produits

Creez une API complete pour gerer des produits avec :

Modele Pydantic Produit (id, nom, prix, stock, categorie)

POST /produits -> creer un produit

GET /produits -> lister tous (avec filtre optionnel par categorie)

GET /produits/{id} -> obtenir un produit

PUT /produits/{id} -> modifier un produit

DELETE /produits/{id} -> supprimer (erreur 404 si inexistant)

Indice : Utilisez une liste comme base de donnees en memoire. Gerez les erreurs avec `HTTPException`.

★★★ Exercice DIFFICILE — API avec Authentification

Creez une API avec un systeme d'authentification simple par tokens :

POST /register -> creer un compte (nom, email, mot de passe)

POST /login -> authentifier et retourner un token (UUID genere avec `uuid.uuid4()`)

GET /profil -> retourne le profil (token requis dans les headers)

POST /logout -> invalider le token

Middleware : verifier le token pour les routes protegees avec `Depends()`

Indice : Stockez les tokens dans un dict `{token: user_id}`. Utilisez `Header` pour lire les headers.

PROJET DE FIN DE CHAPITRE — API REST Complete — Gestionnaire de Taches (Todo App)

Description

Construisez une API FastAPI complete et fonctionnelle pour une application de gestion de taches, prete a etre connectee a un frontend.

Objectifs

41. Modeles : Tache (id, titre, description, statut, priorite, date_creation, date_limite)
42. CRUD complet avec validation Pydantic stricte
43. Filtres : par statut (todo/en_cours/termine), par priorite, par date
44. Recherche par texte dans le titre et la description
45. Statistiques : GET /stats -> nombre par statut, taux de completion, taches en retard
46. Persistance : sauvegarder et charger depuis un fichier JSON

Bonus

Documentez votre API avec des descriptions dans chaque route (parametre description de @app.get). Ajoutez des exemples dans vos modeles Pydantic avec class Config.

La Suite de votre Parcours

Felicitations ! En completant ce guide, vous avez acquis des bases solides en Python et fait vos premiers pas dans le developpement web avec FastAPI. Voici votre feuille de route pour continuer :

Etape	A apprendre	Ressources
Maintenant	Consolidez les 9 chapitres, completez tous les projets	<i>Recodez chaque projet de zero sans regarder les solutions</i>
Prochain	FastAPI avance : bases de donnees SQLite avec SQLAlchemy, authentication JWT	<i>docs.sqlalchemy.org, fastapi.tiangolo.com</i>
Dans 2 mois	Django : ORM, admin, templates, Django REST Framework	<i>docs.djangoproject.com</i>
Dans 4 mois	Flask : microframework, Jinja2, Flask-SQLAlchemy	<i>flask.palletsprojects.com</i>
Bonus	Docker, PostgreSQL, deploiement sur Render ou Railway	<i>docker.com, render.com</i>

TIP

Le secret pour progresser rapidement : CODER TOUS LES JOURS, meme 30 minutes. La theorie sans pratique ne sert a rien. Faites les exercices, les projets, cherchez sur Google.
Rejoignez la communaute Python francophone sur Discord et Reddit (r/learnpython).
Consultez la documentation officielle : docs.python.org/fr et fastapi.tiangolo.com

Bonne chance dans votre parcours Python et developpement Web !